file-based routing structure. While you can technically inspect the structure of these child components, there's no need to manipulate them directly within the `RootLayout`. Next.js will handle the rendering of these components based on the defined routes and their corresponding components.

2.8.6 Route API handlers

Finally, we explore how the UI communicates with the backend in Next.js. Since we don't spin up a new server to handle the API processing of the requests and further communication with the Gemini/OpenAI endpoint, we rely on Next.js route handlers.

Next.js route handlers are equivalent to Express.js handlers with some distinct differences:

- *Location*—Next.js route API handlers reside within an api directory of the Next.js project, using route.js or route.ts files—for example, app/api/route.js or app/blog/api/route.js. We can define multiple handlers that can reside in different route paths or segments. This allows for improved code organization due to the separation of concerns within the Next.js structure.
- *Extended handlers*—Next.js route APIs provide built-in access to `NextRequest` and `NextResponse` objects, which are enhanced versions of the Node.js `http.ClientRequest` and `http.ServerResponse` objects, offering convenient functionalities for handling cookies, headers, and revalidation.
- *Convention*—Next.js route API handlers must be specifically named as route.js or route.ts to work.

Code listing 2.7 shows the equivalent API handler that we use to handle all the backend logic (app/(chat)/api/route.js). I've omitted the part that is common between the Express.js and the Next.js implementation for readability; it can be found in the (chat)/api folder in the respective example project.

Listing 2.7 Handling the API backend logic

```
import { GoogleGenerativeAI } from "@google/generative-ai";

export const dynamic = 'force-dynamic';

const openai = new GoogleGenerativeAI (
        process.env.GEMINI_API_KEY || ''
        );

export async function POST(req) {.
  const { text } = await req.json();
  // Code to request completion data from Google.
  return Response.json({ message });
}
```

Forces dynamic mode for this route handler to prevent caching

Initializes the Gemini API client using the API key from environment variables

Exports the POST request handler function

Parses the request body as JSON to extract the user's text input

Calls the Google chat. completions API. This code is the same as the Express.js handler implementation (lines 30-49 in server.js).

Returns a JSON response containing the formatted message object